

ARIA：一种基于依赖图的检索与迭代自动形式化智能体

Hanyu Wang^{1*} Ruohan Xie^{1*} Yutong Wang^{1*} Guoxiong Gao^{1*} Xintao Yu²

Bin Dong^{3,4,5†}

¹Peking University ²Renmin University of China

³Beijing International Center for Mathematical Research and the New Cornerstone Science Laboratory, Peking University

⁴Center for Machine Learning Research, Peking University

⁵Center for Intelligent Computing, Great Bay Institute for Advanced Study, Great Bay University

wanghanyu2004@stu.pku.edu.cn, awaken.nox@stu.pku.edu.cn

wangyutong25@stu.pku.edu.cn, samggx@stu.pku.edu.cn

1472388812@qq.com, dongbin@math.pku.edu.cn

摘要

定理陈述的准确自动形式化，对于推动研究级数学的自动发现与验证至关重要；然而，由于幻觉、语义不匹配，以及无法合成新定义，大语言模型在这一任务上仍存在主要瓶颈。为了解决这些问题，我们提出 Aria (**A**gent for **R**etrieval and **I**terative **A**utoformalization)，这是一个面向 Lean 中猜想级形式化的系统。它通过两阶段 Graph-of-Thought 过程模拟人类专家推理：先递归地将陈述分解为依赖图，再从已 grounding 的概念出发构造形式化结果。为保证语义正确性，我们引入 **AriaScorer**，它从 Mathlib 检索定义并进行术语级 grounding，从而支持严格而可靠的验证。我们在多类基准上评估 Aria。在 ProofNet 上，它达到 91.6% 的编译成功率和 68.5% 的最终准确率，超过既有方法。在 FATE-X 这一来自研究文献的高难代数问题集合上，它以 44.0% 对 24.0% 的最终准确率超过最佳基线。在同调猜想数据集上，Aria 达到 42.9% 的最终准确率，而所有其他模型均为 0%。

1 引言

近年来，Coq (Barras et al., 1999)、Isabelle (Paulson, 1994) 和 Lean (Moura & Ullrich, 2021) 等交互式定理证明器 (ITP) 已经成为形式化数学的重要生态。其中，Lean 4 及其完整的 Mathlib 库 (mathlib Community, 2020) 正在开创一种新的形式化范式。然而，这一生态的持续增长始终受到巨大人工工作量和深厚专业知识要求的限制。为此，研究社区转向大语言模型 (LLM) 来进行自动形式化，即把非形式化（或自然语言）数学陈述与证明翻译成对应的形式化表达。虽然陈述形式化和证明形式化相互关联，但准确的陈述形式化是基础性的第一步。一个正确形式化的陈述是任何有效证明的前提；它本身也是数学生态中的重要资产，能支持更好的搜索、集成和验证。因此，尽管证明自动化已有进展 (Ren et al., 2025; Chen et al., 2025)，初始陈述翻译的保真度仍是关键瓶颈。LLM 生成的形式化陈述不仅常有编译错误，还常包含更隐蔽的语义缺陷；当形式化对象变成更复杂的研究级或猜想级陈述时，这一挑战更加严重。

这些基础性缺陷会在下游造成几类关键失败。不忠实的翻译会使大规模数据生成流水线偏离目标，让系统在错误前提下消耗大量计算预算。例如，现代证明器常把复杂证明分解成较小的非形式化引理，再逐个翻译并证明。在这一流程中，只要某个引理的翻译有误，不仅整个证明结构会失效，也会污染这一过程中生成的数据集；而这些数据集对未来模型微调至关重要。进一步地，当研究社区推动形式化模型自主探索猜想级问题时，系统如果不能创建并使用必要且通常未见过的 premise（即定义、引理、定理等），就会在任务开始处受阻。缺少这种能力的系统，在这类雄心勃勃的任务上必然会失败。本文通过提出一种稳健方法来生成、迭代并验证形式化陈述，用自动化结构推理处理这些基础瓶颈。

*同等贡献。

†通讯作者。

一个主要挑战来自 LLM 预训练知识的静态性和内在不可靠性。尽管已有基础工作表明 LLM 在本科数学层面具有潜力 (Gao et al., 2024b; Wang et al., 2025), 但当面对研究级陈述时, 这些方法会出现关键失败模式: LLM 容易幻觉, 并依赖过时的预训练知识。它们会生成无效代码, 其中的函数可能并不存在于 Mathlib 中, 或者不兼容快速演化的库工具链。为此, 我们集成 Retrieval-Augmented Generation (RAG) 框架, 通过动态查询最新版本的 Mathlib 库来 grounding 形式化过程, 减轻模型对静态知识的依赖, 并保证与持续演化的工具链兼容。

除知识检索外, 更深层的挑战在于合成。研究级数学本质上涉及创建新的数学对象和定义; 一步式生成方法即便加入检索, 也无法自发合成库中不存在的概念定义。为此, 我们开发了一个由 Graph-of-Thought (GoT) 形式化过程驱动的智能体流水线。该方法模拟专家数学家的工作流: 递归地分解定义依赖, 直到它们被良好 grounding; 随后按自底向上的顺序合成形式化陈述, 直到主目标被形式化。为保证这一过程稳健, 我们在每个节点使用 compiler-in-the-loop 的反思机制。

一旦陈述生成并通过编译检查, 最终挑战就是保证它的语义正确性。虽然 LeanScorer (Xuejun et al., 2025) 等已有方法通过细粒度比较推进了语义检查, 但由于依赖表面文本相似性, 它们无法发现形式术语与非形式术语之间细微的定义性差异。为克服这一限制, 我们提出 AriaScorer, 这是一种增强的语义检查器, 加入了术语级 grounding 步骤。AriaScorer 从 Mathlib 检索所有 Lean 术语的权威定义, 并把这些形式化上下文注入比较过程, 从而实现更严格、更准确的评估。

借助这一经过验证的检查器, 我们在一组研究级数据集上评估 Aria 的端到端性能。我们度量最终准确率, 即生成的形式化陈述同时通过编译检查和语义正确性检查的比例。结果表明, 相比已有工作, Aria 有显著跃升: 在 ProofNet 上达到 68.5%, 并在其他基准上超过先前最优模型, 包括 FATE-H (71.0% 对 43.0%) 和 FATE-X (44.0% 对 24.0%)。尤其值得注意的是, 在一组真实数学猜想构成的挑战性数据集上, 所有基线模型均为 0%, 而 Aria 达到 42.9% 的成功率, 展示出研究级形式化的独特能力。

本文的主要贡献如下:

- 我们提出 Aria, 这是一个陈述自动形式化智能体, 通过整合检索增强生成、graph-of-thought 规划和编译器引导的自反思机制, 模拟人类形式化过程; 该方法尤其适合猜想级问题。
- 我们开发了一个术语级 grounding 的语义评分器 AriaScorer, 用于检测非形式化陈述与 Lean 术语之间的细微差异, 并准确验证形式化结果的数学正确性。
- 我们取得了新的最优性能, 相较先前方法有显著提升: ProofNet 上达到 68.5%, FATE-H 上达到 71.0%, FATE-X 上达到 44.0%, 在数学家提出的真实猜想上达到 42.9%。

本文其余部分组织如下。第 2 节回顾相关工作。第 3 节详细介绍我们提出的方法, 包括 Aria 的架构和核心组件。第 4 节呈现实验结果及其分析。最后, 第 5 节总结全文。

2 相关工作

自动形式化 大语言模型的快速发展推动了自动形式化的显著进展。早期工作通过 few-shot in-context learning (ICL) 展示了成功案例 (Wu et al., 2022; Patel et al., 2024; Zhou et al., 2024)。随着 Lean 社区壮大、Mathlib 库变得更加完备, 大规模数据集的可得性使得通过监督微调 (SFT) 开发专用模型成为可能 (Azerbayev et al., 2023; Jiang et al., 2023; Gao et al., 2024b; Wang et al., 2025)。近期, 强化学习 (RL) 在数学和推理中显示出潜力, 多项工作利用 RL 训练提升自动形式化质量 (Xuejun et al., 2025; Huang et al., 2025)。同时, 其他方法聚焦于提升生成过程本身的质量与可靠性。随着 Lean 生态内搜索能力日益增强, Retrieval-Augmented Generation (RAG) 被证明能有效为模型提供来自庞大 Mathlib 库的相关定义和定理 (Lu et al., 2025)。与此同时, Process-Supervised Verification (PSV) 等新方法利用编译器的直接反馈来引导模型学习过程, 显著提高生成形式化结果的正确性和可靠性 (Lu et al., 2024)。

语义检查 随着陈述自动形式化方法变得更加复杂和多样, 建立可信评估方式来判断形式化陈述在多大程度上保留了非形式化对应物的数学含义, 变得至关重要。人类专家当然能提供可靠的一致性评估 (Azerbayev et al., 2023), 但随着陈述更加复杂, 这类评估也越来越困难。因此, perplexity (Wang et al., 2018) 和 BLEU (Wang et al., 2018; Azerbayev et al., 2023) 被用作代理指标。另一种常见做法是让 LLM 将有效的形式化陈述反向翻译成非形式化陈述, 再使

用另一个 LLM 评估语义保留情况 (Ying et al., 2024; Gao et al., 2024b; Liu et al., 2025b)。此外, 也有工作引入 LLM judge 一致投票与 Lean 专家验证结合的结构, 并将其作为训练中的奖励信号 (Wang et al., 2025)。还有工作考虑对非形式化陈述进行子任务分解, 从而更细粒度地过滤错误形式化结果 (Xuejun et al., 2025)。近期, 一种用于判断两个形式化陈述数学等价性的自动神经符号方法被广泛采用。该方法使用语义保持 tactic; 当且仅当存在形式证明能连接两个陈述时, 才确认二者等价 (Liu et al., 2025a; Wu et al., 2025)。

3 方法

本节详细介绍我们的方法, 它由两个主要组件构成。整体流水线如图 1 所示。第 3.1 节描述 Aria 的架构: 这是一个结构化流水线, 用于处理猜想级数学陈述中的深层概念依赖。第 3.2 节介绍我们集成的语义检查器, 用于验证智能体输出是否忠实于原始数学意图。

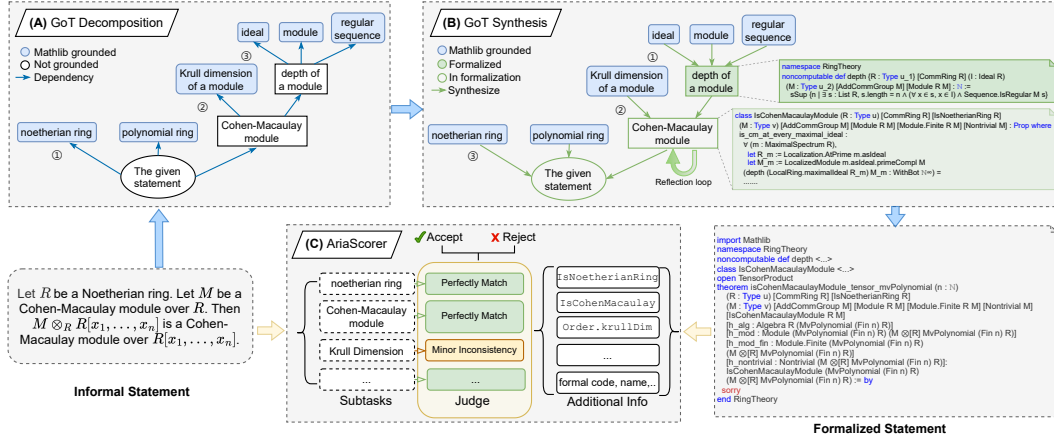


图 1: Aria 系统的整体流水线。(A) **Graph-of-Thought 分解**: Aria 将非形式化陈述展开为概念依赖图, 并在 Mathlib 中对概念进行 grounding。(B) **Graph-of-Thought 合成**: 系统在依赖图引导下执行自底向上的合成过程, 并纳入自反思循环。(C) **AriaScorer**: 专门验证生成的形式化陈述与原始非形式化陈述之间语义正确性的模块。

3.1 GRAPH-OF-THOUGHT (GoT) 自动形式化流水线

本节详细说明我们的智能体 Aria 的架构。该架构超越了传统的直接一步生成方法; 这些方法在复杂的猜想级陈述上往往失败。如图 1 所示, 我们的智能体通过结构化流水线运行, 系统地分解、解析、合成并验证形式化结果, 从而模拟人类数学家的方法化过程。

该流水线使用 Graph-of-Thought (GoT) 规划器将非形式化陈述分解为概念图, 其中每个概念节点表示一个定义、结构或类, 如图 1 (A) 所示。图中的每个概念节点随后由 grounding 模块处理; 该模块采用由 LeanSearch (Gao et al., 2024a) 支持的 Retrieval-Augmented Generation (RAG) 框架, 将已知概念锚定到 Mathlib 库。对于未 grounding 的概念, 合成模块会自底向上生成新定义, 如图 1 (B) 所示。所有输出都由 compiler-in-the-loop 的反思模块验证并改进。最后, 我们使用检索式检查器来验证语义正确性。

3.1.1 GoT 分解阶段

为管理形式化高难度陈述所需的复杂无环定义与引理依赖图, 我们的智能体架构以基于 GoT 范式的规划模块为中心。该模块把单体形式化任务转化为结构化、可管理的工作流, 即把任务建模为概念依赖图的构造与求解, 如图 1 (A) 所示。这一方法基于数学抽象的一条关键原则, 而我们的智能体直接利用了这一原则: 任何概念无论多么复杂, 都可以只用其直接前置概念来定义。

规划模块的核心是概念依赖图, 这是一种动态数据结构, 作为智能体的工作记忆。该图由概念节点和有向边组成, 每个节点表示最终形式化所需的一个数学概念。

对于给定陈述，Aria 启动完整形式化例程：它自顶向下展开概念图依赖，直到所有叶子节点都能在 Mathlib 中 grounding。为实现这一 grounding，智能体在每个节点查询 LeanSearch。LeanSearch 是一个专门搜索引擎，其索引会持续更新以反映 Mathlib 的近期版本，因此在 Mathlib 演进时仍能保持有效。检索过程返回来自 Mathlib 的候选项排序列表，每个候选项包含形式化陈述及其对应的非形式化描述，并按与输入概念名称的语义相关性排序。

由于排名最高的搜索结果并不总是形式化所需的规范定义，智能体使用 LLM 作为精细推理器来分析检索候选项，识别最合适的单一规范定义。如果推理器判断候选项中没有合适匹配（即该概念未在 Mathlib 中 grounding），则该节点被视为依赖图中的内部节点（如图 1 (A) 所示）。其未解析状态会触发规划器继续自顶向下展开其子节点，随后该节点被标记为待合成。

3.1.2 GoT 合成阶段

所有展开完成后，智能体立即转入整个图的自底向上合成阶段，如图 1 (B) 所示。对于任何无法在 Mathlib 库中 grounding 的概念，都会调用合成模块（例如如图 1 (B) 中的“Cohen-Macaulay Module”概念）。该模块负责从基础开始生成可验证的形式化定义，并由稳健的 compiler-in-the-loop 反思过程引导，以保证句法正确性。

对于给定目标概念，智能体首先收集其所有直接依赖项（即依赖图中的子节点）的已验证形式化代码，作为上下文交给 LLM，以生成该目标的 Lean 形式定义。生成的代码会立即送入 Lean 编译器进行句法检查。若编译失败，错误信息和失败代码将作为反馈返回给 LLM 进行修正。若编译成功，该代码被标记为已合成，并用于其父节点的合成。

虽然这一过程能保证句法有效，但不能排除“类型正确但语义错误”的翻译。为了用更灵活的方法检查代码的语义正确性，我们的方法加入了增强的检索式语义一致性检查器，详见第 3.2 节。

3.2 语义正确性模块：ARIASCORER

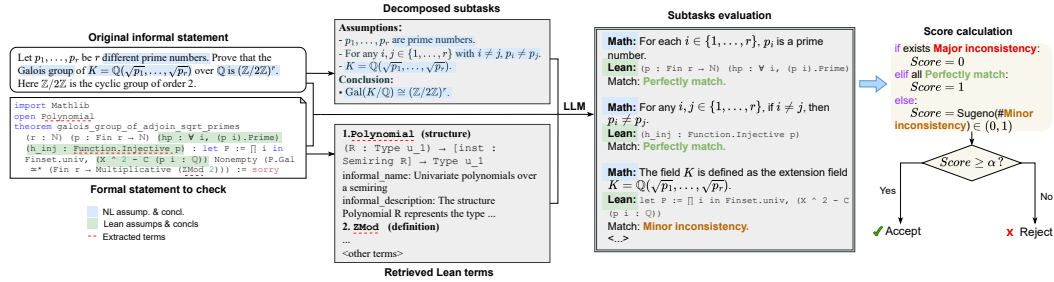


图 2: AriaScorer 的整体流水线：非形式化陈述被分解为子任务，通过检索得到的 Lean 术语进行 grounding，再把各项评估聚合为最终分数，并与阈值 $\alpha \in [0, 1]$ 比较以得到二元决策。

3.2.1 基础：LEANSORER

我们提出一种针对自动形式化 Lean 陈述的语义正确性检查器，目标是缓解幻觉并减少 LLM 生成输出中固有的 false positive。为处理高难度陈述（例如猜想）中条件密集、假设敏感的特点，我们采用 LeanScorer (Xuejun et al., 2025) 的子任务分解策略，通过子任务级比较评估语义正确性。

给定原始非形式化陈述，LLM 将其分解成原子假设和结论，然后对每个子任务进行评估，判断其形式化子句与对应非形式化内容的匹配程度。子任务被标记为 Perfectly Match、Minor Inconsistency 或 Major Inconsistency，这些标签通过 fuzzy integral 聚合成 0 到 1 之间的最终分数，其中 0 表示存在重大错误，1 表示所有子任务完全对齐。除这两种情形外，分数会随着 minor inconsistency 的累积而从 1 逐渐下降，刻画细微偏差的累积影响。系统再应用可调节阈值进行二元决策，在容忍小偏差和拒绝语义错误形式化之间取得平衡。然而，由于该方法仍在很大程度上依赖表面文本相似性，它容易受到隐藏在表面相近表达之下的语义错配影响，这也促使我们引入术语 grounding 的评估模块。

3.2.2 术语级语义 GROUNDING

为确保评估过程与形式 Lean 陈述的真实语义对齐，我们引入一个新步骤：**术语级检索与解释**。在这一步中，我们使用 Lean 静态分析器 `jixia`¹，抽取形式化陈述中引用的每个 Lean 术语，并查询 Herald (Gao et al., 2024b) 建立的 curated informalized Mathlib 数据集，以检索每个术语的名称、类别、类型、值、非形式化名称和非形式化描述。随后，检索到的术语信息会与原始非形式化和形式化陈述、分解后的子任务列表、few-shot 示例一起提供给 LLM，作为子任务评估阶段的上下文。

这一过程构成了**语义 grounding**的基础，使 AriaScorer 能够基于形式化组件的真实含义进行推理，而不是依赖表面语法。因此，AriaScorer 能识别细微不一致，例如反转的参数顺序或非预期的类型强制转换，而这些错误都很容易被纯文本比较漏掉。该步骤有助于防止常见 LLM 失败模式，包括：(i) 忽略 Lean 术语定义中隐含的前置条件或约束；(ii) 在 Lean 定义与更熟悉的数学对应物不同的时候，默认按熟悉概念误读 Lean 定义；(iii) 幻觉式地给出 Lean 术语的错误解释。这些错误类型及 AriaScorer 的处理方式见第 4.3.3 节，详细例子见案例研究。

通过把评估 grounding 到 Lean 术语的真实语义上，AriaScorer 在涉及新引入或用户自定义结构的场景中特别能提供可靠、更细粒度的评估。为验证语义 grounding 步骤的影响，我们在第 4.3 节给出消融实验，展示其在错误检测和 false positive 降低方面的明确收益。

4 实验

我们进行了广泛实验来评估 Aria 和 AriaScorer 的性能。第 4.1 节描述 Aria 的实验设置，第 4.2 节呈现主要结果。第 4.3 节展示用于验证 AriaScorer 的综合实验。最后，第 4.4 节通过消融研究分析关键组件的贡献。

4.1 ARIA 的实验设置

本节概述用于严格评估 Aria 性能的实验框架，包括使用的数据集和比较基线。

4.1.1 基准

为严格评估智能体在不同难度和问题类型上的表现，我们在一组基准上进行评估。具体而言，我们使用广泛采用的 ProofNet (Azerbaiyev et al., 2023) 来保证泛化性和与既有工作的可比性，并使用 FATE (Jiang et al., 2025) (Formal Algebra Theorem Evaluation) 集合以及包含 14 个真实猜想的数据集，以测试复杂研究级问题上的性能。

ProofNet 为评估泛化性，我们使用 ProofNet，这是一个被广泛采用的本科数学基准。它保证我们的复杂智能体架构不仅对复杂猜想有效，也能在标准问题上保持稳健且具有竞争力。

FATE-H 与 FATE-X FATE 集合用于在高等数学上测试智能体。FATE-H 包含代数期末考试问题，FATE-X 包含来自博士资格考试和研究文献的更难问题。选择这些基准，是为了评估智能体处理复杂研究级数学的能力。

交换代数中的同调猜想 (Conjectures) 最后，我们在 Melvin Hochster 汇编的一组交换代数中的 14 个真实 Homological Conjectures (Wikipedia contributors, 2025) 上测试 Aria。这些猜想探测交换 Noetherian 环的同调性质与其结构特征之间的深层联系，直接而有挑战性地检验 Aria 形式化活跃数学研究问题的能力。

4.1.2 基线模型

为评估智能体架构的有效性，我们将其与若干领先的陈述自动形式化模型比较，包括强推理模型 Gemini-2.5-Pro (Google DeepMind)，以及专门自动形式化模型 Goedel-Formalizer-V2-32B (Goedel-V2) (Lin et al., 2025)、Kimina-Autoformalizer-7B (Kimina) (Wang et al., 2025) 和 Herald-translator (Herald) (Gao et al., 2024b)。

¹<https://github.com/frenzymath/jixia>

表 1: Aria 与专用模型的端到端自动形式化结果比较。所有数值均为成功率 (%)；我们报告编译成功率和更严格的最终准确率 (同时通过编译和 AriaScorer 语义检查)。Conjectures 数据集的结果经过人工验证。Kimina 在 ProofNet 上的分数因潜在数据污染而标记*。

方法	ProofNet		FATE-H		FATE-X		Conjectures
	Compiler	Final acc.	Compiler	Final acc.	Compiler	Final acc.	
Aria	91.6	68.5	89.0	71.0	69.0	44.0	42.9
Goedel-V2 (pass@16)	–	–	77.0	–	37.0	–	0
Goedel-V2 (pass@32)	–	–	82.0	–	49.0	–	0
Goedel-V2 (pass@64)	–	–	88.0	–	58.0	–	0
Goedel-V2 (pass@128)	–	–	91.0	43.0	63.0	24.0	0
Gemini-2.5-Pro (pass@1)	55.8	27.8	35.0	31.0	27.0	21.0	0
Goedel-V2 (pass@1)	59.6	32.0	35.0	27.0	27.0	16.0	0
Kimina (pass@1)	70.4*	24.7*	10.0	0.0	5.0	1.0	0
Herald (pass@1)	48.5	18.3	24.0	12.0	8.0	5.0	0

* Kimina 在 ProofNet 数据集上训练过，因此其报告分数可能无法反映真实泛化能力。

4.2 主要结果与分析

为评估模型性能，我们在第 4.1 节所述基准上进行了与基线的综合比较。如表 1 所示，我们的智能体在所有评估中都表现突出。

如表 1 所示，Aria 在每个基准上都明显优于所有基线。然而，Aria 中的 GoT 和反思机制会为每个翻译任务调用多次 LLM。为公平比较效率，必须同时考虑成功率和计算成本；我们以每个问题的 API 调用次数作为主要成本指标。由于 Goedel-V2 是单次采样中表现最好的专用模型，其结果与 Gemini-2.5-Pro 基线相当，我们选择它与 Aria 进行计算预算的直接比较。首先我们确定，Aria 在 FATE-X 基准上平均每题需要 17.7 次调用。

基于此，我们为 Goedel-V2 设计了一系列实验，从 pass@16 到 pass@128。如表 1 所示，虽然 Goedel-V2 的编译率随调用次数增加而提高，其最终准确率仍低于 Aria。即使与使用超过 7 倍调用次数的 Goedel-V2 pass@128 相比，Aria 仍保持更高最终准确率。

最重要的是，我们在 Conjectures 数据集上的比较分析揭示了 Aria 取得突破性表现的原因。通过对生成代码的综合案例研究，我们识别出基线模型的不同缺陷：大型推理模型由于缺少 Mathlib 专家知识，倾向于幻觉错误接口；而专用自动形式化模型缺少管理猜想级概念依赖所需的数学推理能力，这从它们倾向于简单复制训练数据格式而没有真正理解底层数学逻辑中可见。Aria 的架构在强推理模型之上整合 GoT 与检索模块，成功解决了这两类限制。更多形式化猜想案例见附录 A。

4.3 ARIASCORER 的验证

4.3.1 实验设置

我们在 FATE-X 基准上将语义正确性检查器与领先替代方法进行比较。评估使用 Aria 智能体生成的句法正确自动形式化输出。该基准包含复杂数学陈述和高级定义，为语义精度提供严格测试。

Ground truth 数据集构造 我们构造专家验证的 ground truth 数据集，根据数学保真度把每个形式化结果标为“True”或“False”。标注由一名纯数学代数方向博士候选人提供，该标注者也参与过 Mathlib 贡献；随后由具有相同资质的第二位专家独立验证。之后我们使用这一数据集评测多种语义正确性检查器。

基线 我们将 AriaScorer 与若干已有语义正确性检查方法比较。第一种是 LeanScorer (Xuejun et al., 2025)，这是一种使用分解与匹配的方法；由于原始版本不开源，我们重新实现了它。我们的 LeanScorer 重新实现同时也是 AriaScorer 的关键消融，代表没有术语级 grounding 步骤的完整流水线。第二种是 Back Translation (Ying et al., 2024; Gao et al., 2024b)，即把形式化陈述反向翻译为非形式化陈述，再用 LLM 判断相似性。为进行受控比较，AriaScorer、LeanScorer 和 BackTranslation 都基于相同基础模型 Gemini-2.5-Pro。我们还直接评估 Gemini-2.5-Pro 在

该任务上的表现。这样的比较框架确保 AriaScorer 的准确性提升来自新的术语级分析，而不是底层语言模型差异。

评估指标 我们使用二分类评估，其中形式化结果被标为正例（正确）或负例（有缺陷）。性能基于 True Positives (TP)、True Negatives (TN)、False Negatives (FN) 和 False Positives (FP) 计数。例如，False Positive 指检查器错误地批准了有缺陷的形式化结果。随后用这四类结果计算并报告 accuracy、precision、recall 和 F1 score。

4.3.2 ARIASCORER 的性能

表 2: 不同语义正确性检查器的性能比较。实验在 Aria 于 FATE-X 上的自动形式化输出上进行。二元决策的分数阈值记为 α 。

	AriaScorer ($\alpha = 0$)	AriaScorer ($\alpha = 0.9$)	LeanScorer ($\alpha = 0$)	LeanScorer ($\alpha = 0.9$)	Back Translation	Gemini
TP	50	42	46	44	7	45
TN	12	15	3	7	16	8
FP	5	2	14	10	1	9
FN	2	10	6	8	45	7
Accuracy	89.9%	82.6%	71.0%	73.9%	33.3%	76.8%
Precision	90.9%	95.5%	77.6%	81.5%	87.5%	83.3%
Recall	96.2%	80.8%	88.5%	84.6%	13.5%	86.5%
F1	93.5%	87.5%	82.1%	83.0%	23.3%	84.9%

在检查 Aria 于 FATE-X 上输出的语义正确性方面，AriaScorer 是表现最好的模型。当阈值 $\alpha = 0$ 时，它取得最高 accuracy (89.9%)、recall (96.2%) 和 F1 score (93.5%)，显著超过所有基线。相较 LeanScorer，其 precision 与 recall 的优势凸显了术语级 grounding 的收益。将阈值提高到 $\alpha = 0.9$ 后，AriaScorer 的 precision 提升到最高 95.5%。这体现出关键权衡：较低阈值更能容忍数学等价表达，从而最大化 recall；较高阈值则施加更严格标准，适合真实部署中最小化 false positive。相比之下，Back Translation 基线要求精确文本匹配，因此 precision 很高但整体 recall 很低。虽然我们在其他所有实验中采用 $\alpha = 0.9$ 的高精度设置， $\alpha = 0$ 的结果最能展示术语 grounding 方法的根本优势。

4.3.3 ARIASCORER 的关键发现

通过引入术语级 grounding，AriaScorer 解决了语义正确性检查中的常见失败模式。我们的消融研究强调了它的三项关键优势：

隐含语义包含 通过检索形式术语在 Mathlib 中的完整定义，AriaScorer 能识别其中包含的任何隐含前置条件或约束。这揭示了准确评估所需、纯文本比较会忽略的重要依赖。（见附录 B.1。）

定义差异检测 AriaScorer 能检测形式术语精确定义与非形式化概念预期含义之间的细微差异。通过把检索到的 Mathlib 定义与原题上下文比较，它能识别 Lean 术语虽然文本相近、却具有不同数学解释的情况。（见附录 B.2。）

通过 Grounding 抑制幻觉 AriaScorer 通过 grounding 评估过程来抑制 LLM 幻觉。在调用 LLM 前，它把所有形式术语的权威 Mathlib 定义注入 prompt。这约束模型只能基于已验证的 ground truth 推理，确保其输出反映形式化代码的真实语义。（见附录 B.3。）

4.4 消融研究摘要

我们进行了一系列综合消融研究，以量化 Aria 核心组件的独特贡献：Reflection 机制、Graph-of-Thought (GoT) 规划器和 Retrieval-Augmented Generation (RAG) 模块。我们的发现，尤其是在挑战性的 Conjectures 数据集上，显示三者都不可或缺。

- 移除 Reflection 模块会使 FATE-X 和 Conjectures 上的性能崩塌，证明它对于获得正确代码是必要的。

- 移除 GoT 规划器会削弱智能体形式化新概念的能力，使成功猜想数从 6 个降为 1 个。这凸显了它在施加逻辑结构方面的关键作用。此外，我们发现 GoT 模块的影响在更具挑战性的数据集上更加明显。
- 禁用 RAG 模块会使 Conjectures 上成功率完全降为 0%，确认它在 grounding 智能体并防止不存在概念的基础性幻觉方面至关重要。

详细流程和分析见附录 C。

5 结论

本文提出 Aria，这是一个整合检索增强生成、graph-of-thought 规划和自反思机制的陈述自动形式化智能体。该集成方法使 Aria 成为首个能够自主合成复杂新定义的智能体，而这些定义正是形式化猜想等高难数学陈述所需的。该能力直接解决了先前方法的核心限制：在遇到未见概念时，它们会因幻觉和逻辑错误失败。此外，我们提出了新的语义正确性检查器 AriaScorer，它从 Mathlib 检索定义并用于术语级 grounding，从而实现严格而可靠的验证。

我们的综合实验结果表明，该智能体在从本科水平到猜想的不同难度基准上，都取得领先最终准确率。这一成功在高度挑战性的 Homological Conjectures 数据集上尤为明显，智能体取得了突破性表现。

鉴于陈述形式化是定理证明的关键前提，我们对猜想级陈述的成功形式化为未来自动数学证明研究奠定了坚实基础。

致谢

本工作部分得到中国国家重点研发计划 2024YFA1014000、新基石研究员项目和 Ubiquant 的支持。

参考文献

- Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W Ayers, Dragomir Radev, and Jeremy Avigad. Proofnet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint arXiv:2302.12433*, 2023.
- Bruno Barras, Samuel Boutin, Cristina Cornes, Judicaël Courant, Yann Coscoy, David Delahaye, Daniel de Rauglaudre, Jean-Christophe Filliâtre, Eduardo Giménez, Hugo Herbelin, et al. The Coq proof assistant reference manual. *INRIA*, 1999.
- Luoxin Chen, Jinming Gu, Liankai Huang, Wenhao Huang, Zhicheng Jiang, Allan Jie, Xiaoran Jin, Xing Jin, Chenggang Li, Kaijing Ma, Cheng Ren, Jiawei Shen, Wenlei Shi, Tong Sun, He Sun, Jiahui Wang, Siran Wang, Zhihong Wang, Chenrui Wei, Shufa Wei, Yonghui Wu, Yuchen Wu, Yihang Xia, Huajian Xin, Fan Yang, Huaiyuan Ying, Hongyi Yuan, Zheng Yuan, Tianyang Zhan, Chi Zhang, Yue Zhang, Ge Zhang, Tianyun Zhao, Jianqiu Zhao, Yichi Zhou, and Thomas Hanwen Zhu. Seed-Prover: Deep and Broad Reasoning for Automated Theorem Proving, August 2025. URL <http://arxiv.org/abs/2507.23726>. arXiv:2507.23726 [cs].
- Guoxiong Gao, Haocheng Ju, Jiedong Jiang, Zihan Qin, and Bin Dong. A semantic search engine for mathlib4, 2024a. URL <https://arxiv.org/abs/2403.13310>.
- Guoxiong Gao, Yutong Wang, Jiedong Jiang, Qi Gao, Zihan Qin, Tianyi Xu, and Bin Dong. Herald: A natural language annotated lean 4 dataset. *arXiv preprint arXiv:2410.10878*, 2024b.
- Google DeepMind. Gemini 2.5 pro. <https://deepmind.google/technologies/gemini/pro/>, 2025.
- Yanxing Huang, Xinling Jin, Sijie Liang, Peng Li, and Yang Liu. Formarl: Enhancing autoformalization with no labeled data. *arXiv preprint arXiv:2508.18914*, 2025.
- Albert Q Jiang, Wenda Li, and Mateja Jamnik. Multilingual mathematical autoformalization. *arXiv preprint arXiv:2311.03755*, 2023.

-
- Jiedong Jiang, Wanyi He, Yuefeng Wang, Guoxiong Gao, Peihao Wu, Bryan Dai, and Bin Dong. Introducing fate: A multi-level formal benchmark for frontier algebraic problems. <https://frenzymath.com/blog/fate/>, Aug 2025.
- Yong Lin, Shange Tang, Bohan Lyu, Ziran Yang, Jui-Hui Chung, Haoyu Zhao, Lai Jiang, Yihan Geng, Jiawei Ge, Jingruo Sun, Jiayun Wu, Jiri Gesi, Ximing Lu, David Acuna, Kaiyu Yang, Hongzhou Lin, Yejin Choi, Danqi Chen, Sanjeev Arora, and Chi Jin. Goedel-prover-v2: Scaling formal theorem proving with scaffolded data synthesis and self-correction, 2025. URL <https://arxiv.org/abs/2508.03613>.
- Qi Liu, Xinhao Zheng, Xudong Lu, Qinxiang Cao, and Junchi Yan. Rethinking and improving autoformalization: Towards a faithful metric and a dependency retrieval-based approach. In *The Thirteenth International Conference on Learning Representations*, 2025a. URL <https://openreview.net/forum?id=hUb2At2DsQ>.
- Xiaoyang Liu, Kangjie Bao, Jiashuo Zhang, Yunqi Liu, Yuntian Liu, Yu Chen, Yang Jiao, and Tao Luo. Atlas: Autoformalizing theorems through lifting, augmentation, and synthesis of data. *arXiv preprint arXiv:2502.05567*, 2025b.
- Jianqiao Lu, Yingjia Wan, Zhengying Liu, Yinya Huang, Jing Xiong, Chengwu Liu, Jianhao Shen, Hui Jin, Jipeng Zhang, Haiming Wang, et al. Process-driven autoformalization in lean 4. *arXiv preprint arXiv:2406.01940*, 2024.
- Wangyue Lu, Lun Du, Sirui Li, Ke Weng, Haozhe Sun, Hengyu Liu, Minghe Yu, Tiancheng Zhang, and Ge Yu. Automated formalization via conceptual retrieval-augmented llms. *arXiv preprint arXiv:2508.06931*, 2025.
- The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, 2020.
- Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe (eds.), *Automated Deduction – CADE 28*, pp. 625–635, Cham, 2021. Springer International Publishing. ISBN 978-3-030-79876-5.
- Nilay Patel, Rahul Saha, and Jeffrey Flanigan. A new approach towards autoformalization, 2024. URL <https://arxiv.org/abs/2310.07957>.
- Lawrence C Paulson. *Isabelle: A Generic Theorem Prover*. Springer, 1994.
- ZZ Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liye Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, et al. Deepseek-prover-v2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025.
- Haiming Wang, Mert Unsal, Xiaohan Lin, Mantas Baksys, Junqi Liu, Marco Dos Santos, Flood Sung, Marina Vinyes, Zhenzhe Ying, Zekai Zhu, et al. Kimina-prover preview: Towards large formal reasoning models with reinforcement learning. *arXiv preprint arXiv:2504.11354*, 2025.
- Qingxiang Wang, Cezary Kaliszyk, and Josef Urban. First experiments with neural translation of informal to formal mathematics. In *International Conference on Intelligent Computer Mathematics*, pp. 255–270. Springer, 2018.
- Wikipedia contributors. Homological conjectures in commutative algebra — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Homological_conjectures_in_commutative_algebra&oldid=1299704292, 2025. [Online; accessed 22-September-2025].
- Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models, 2022. URL <https://arxiv.org/abs/2205.12615>.

-
- Yutong Wu, Di Huang, Ruosi Wan, Yue Peng, Shijie Shang, Chenrui Cao, Lei Qi, Rui Zhang, Zidong Du, Jie Yan, and Xing Hu. StepFun-Formalizer: Unlocking the Autoformalization Potential of LLMs through Knowledge-Reasoning Fusion, August 2025. URL <http://arxiv.org/abs/2508.04440>. arXiv:2508.04440 [cs].
- Yu Xuejun, Jianyuan Zhong, Zijin Feng, Pengyi Zhai, Roozbeh Yousefzadeh, Wei Chong Ng, Haoxiong Liu, Ziyi Shou, Jing Xiong, Yudong Zhou, et al. Mathesis: Towards formal theorem proving from natural languages. *arXiv preprint arXiv:2506.07047*, 2025.
- Huaiyuan Ying, Zijian Wu, Yihan Geng, Jiayu Wang, Dahua Lin, and Kai Chen. Lean workbook: A large-scale lean problem set formalized from natural language math problems. *Advances in Neural Information Processing Systems*, 37:105848–105863, 2024.
- Jin Peng Zhou, Charles Staats, Wenda Li, Christian Szegedy, Kilian Q. Weinberger, and Yuhuai Wu. Don’t trust: Verify – grounding llm quantitative reasoning with autoformalization, 2024. URL <https://arxiv.org/abs/2403.18120>.

A ARIA 生成陈述案例研究

本节通过若干代表性案例，对 Aria 生成的陈述进行定性分析，以展示其优势与局限。对于每个案例，我们都将智能体的概念依赖图可视化为了“blueprint”——这是 Lean 社区中表示依赖关系的标准可视化风格——以展示其规划过程。随后，我们给出最终形式化结果，并将其与 Goedel-V2-Formalizer-32B 和 Gemini-2.5-Pro 模型在同一问题上的输出比较。

A.1 例 1: KOETHE 猜想

非形式化陈述

设 R 为环。若 R 没有非零 nil ideal (two-sided)，则它也没有非零 nil one-sided ideal (既没有左理想也没有右理想)。

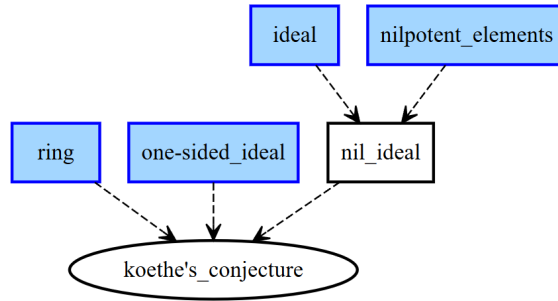


图 3: Koethe 猜想的依赖图

ARIA 生成陈述

```

import Mathlib

universe u

/-- An ideal `I` is a nil ideal if every element of `I` is nilpotent. -/
def IsNil {R : Type u} [Semiring R] (I : Ideal R) : Prop :=
  ∀ x ∈ I, IsNilpotent x

/--
If a ring `R` has no non-zero nil two-sided ideals, then it has no non-zero nil
one-sided ideals (neither left nor right).
-/
theorem no_nil_ideals_implies_no_nil_one_sided_ideals {R : Type u} [Ring R]
  (h : ∀ (I : Ideal R), IsNil I → I = ⊥) :
  (∀ (L : Submodule R R), (∀ x ∈ L, IsNilpotent x) → L = ⊥) ∧
  ∀ (J : Submodule (MulOpposite R) R), (∀ x ∈ J, IsNilpotent x) → J = ⊥ :=
  sorry

```

GOEDEL 生成陈述

```

import Mathlib
import Aesop

set_option maxHeartbeats 0

open BigOperators Real Nat Topology Rat

theorem statement (R : Type*) [Ring R] :
  (∀ I : Ideal R, I ≠ ⊥ → ∃ x ∈ I, ¬IsNilpotent x) →
  (∀ L : Ideal R, L ≠ ⊥ → ∃ x ∈ L, ¬IsNilpotent x) ∧

```

```
( $\forall$  R : Ideal R, R  $\neq$   $\perp$   $\rightarrow$   $\exists$  x  $\in$  R,  $\neg$ IsNilpotent x) := by sorry
```

GEMINI 生成陈述

```
import Mathlib

variable (R : Type*) [Ring R]

def koethe_conjecture : Prop :=
  ( $\forall$  (I : Ideal R), IsNil I  $\rightarrow$  I =  $\perp$ )  $\rightarrow$ 
  (
    ( $\forall$  (J : Submodule R R), IsNil J  $\rightarrow$  J =  $\perp$ )  $\wedge$ 
    ( $\forall$  (K : Submodule Rop R), IsNil K  $\rightarrow$  K =  $\perp$ )
  )
```

分析

这个例子清楚体现了 Aria 相比 Goedel 这样的专门形式化模型以及 Gemini 这样的强通用推理模型的架构优势。

三者中，Gemini 的结果成功保留了原始数学意图，正确识别出需要区分 two-sided ideal 和 one-sided ideal。然而，它由于幻觉了不存在的 API (IsNil) 而无法编译，反映出其强高层推理与库 API 知识之间存在断裂。

相比之下，Goedel 的输出虽然可编译，但语义错误。它没有捕捉到 one-sided ideal 应分别由 Submodule R R 和 Submodule (MulOpposite R) R 表示的非平凡知识，而是把所有理想都形式化为 Ideal R；后者只表示 two-sided ideal，从而使形式化结果完全偏离原始数学意图。

Aria 的形式化同时句法正确且语义正确。它使用正确的 Mathlib 类型来表示不同理想，并用 nilpotent ideal 的前置形式定义 (def IsNil) 提高可读性和模块化。这一成功凸显了 Aria 的 GoT 架构优势：它能够进行高层概念规划，同时严格 grounding 到 Mathlib，从而避免 Gemini 的 API 幻觉和 Goedel 的语义错误。

A.2 例 2: BALANCED BIG COHEN-MACAULAY MODULES 的存在性猜想

非形式化陈述

设 R 为 Noetherian 交换局部环，极大理想为 m_R 。存在一个 R -module W ，使得 $m_R W \neq W$ ，并且 R 的每个 system of parameters 都是 W 上的 regular sequence。

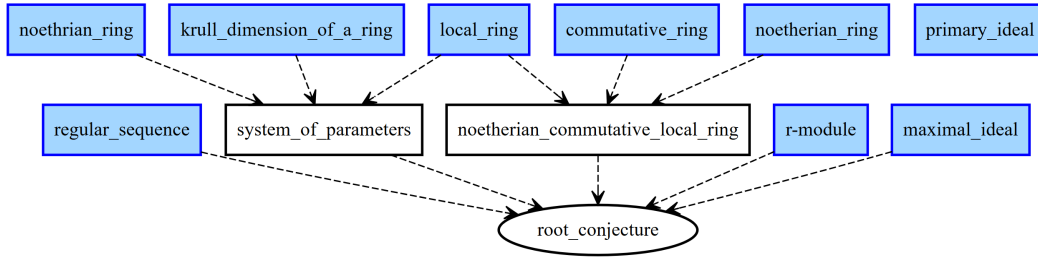


图 4: Balanced Big Cohen-Macaulay Modules 存在性猜想的依赖图

ARIA 生成陈述

```
import Mathlib

/-- A ring `R` is a Noetherian local ring if it is both Noetherian and a local
ring. -/
class IsNoetherianLocalRing (R : Type*) [CommRing R]
  extends IsNoetherianRing R, IsLocalRing R : Prop
```

```

-- Note: `ringKrullDim` is already defined in Mathlib.

-- Note: `Ideal.IsPrimary` is standard in Mathlib and its definition
-- matches the one provided in the context.

/--
In a Noetherian local ring `R` of Krull dimension `d`, a "system of parameters"
is a
sequence of `d` elements `x_1, ..., x_n` in the maximal ideal `m` such that the
ideal
`I = <x_1, ..., x_n>` generated by these elements is `m`-primary (i.e., it is a
primary ideal
whose radical is the maximal ideal `m`).
-/
def IsSystemOfParameters {R : Type*} [CommRing R] [IsNoetherianLocalRing R]
  (d : ℕ) (x : Fin d → R) : Prop :=
  ringKrullDim R = ↑d
  (∀ i, x i ∈ LocalRing.maximalIdeal R) ∧
  (let I := Ideal.span (Set.range x);
   I.IsPrimary ∧ I.radical = LocalRing.maximalIdeal R)

/--
Let  $R$  be a Noetherian commutative local ring with maximal ideal  $m_R$ .
There exists a  $R$ -module  $W$  such that  $m_R W \neq W$  and every system of parameters
for  $R$  is a regular sequence on  $W$ .
-/
theorem exists_module_where_system_of_parameters_is_regular_sequence
  (R : Type*) [CommRing R] [IsNoetherianLocalRing R] :
  ∃ (W : Type*) (AddCommGroup W) (Module R W),
  (LocalRing.maximalIdeal R) · (⊤ : Submodule R W) ≠ ⊤ ∧
  ∀ (d : ℕ) (x : Fin d → R),
    IsSystemOfParameters d x → RingTheory.Sequence.IsRegular W (List.ofFn x)
:= by
  sorry

```

GOEDEL 生成陈述

```

import Mathlib
import Aesop
set_option maxHeartbeats 0
open BigOperators Real Nat Topology Rat
theorem statement (R : Type*) [CommRing R] [IsLocalRing R] [IsNoetherianRing R] :
  ∃ (W : Type*) [AddCommGroup W] [Module R W],
    (maximalIdeal R : Ideal R) · W ≠ W ∧
    ∀ (s : Set R), s.Finite ∧ Ideal.span s = maximalIdeal R →
      ∀ (f : R → W), (∀ r ∈ s, f r ≠ 0) → LinearIndependent R f :=
  by sorry

```

GEMINI 生成陈述

```

import Mathlib

open LocalRing

-- The proposition is defined within a `variable` block,
-- which sets the context for the ring R.
variable (R : Type*) [CommRing R] [IsNoetherianRing R] [LocalRing R]

/--
Formalization of the statement:
"There exists an  $R$ -module  $W$  such that  $m_R \cdot W \neq W$  and every system
of parameters for  $R$  is a regular sequence on  $W$ ."
-/
def module_with_regular_sop_exists : Prop :=

```

```

-- "There exists an R-module W..."
∃ (W : Type*) [AddCommGroup W] [Module R W],
-- "...such that  $m_R * W \neq W$ ..."
(maximalIdeal R) · (T : Submodule R W) ≠ T ∧
-- "...and every system of parameters for R..."
∀ {s : Fin (Ring.Dimension R) → (maximalIdeal R)},
(IsSystemOfParameters s) →
-- "...is a regular sequence on W."
(IsRegularSequence (fun i ↦ (s i : R)) W)

```

分析

这是一个更复杂的猜想，三个生成例子再次凸显了 Goedel 这类专门模型在捕捉复杂数学语义方面的弱点，以及 Gemini 这类通用推理模型对 Mathlib 库知识的缺口。

在该例中，Gemini 使用了库中不存在的定义 `IsSystemOfParameters`。对于 regular sequence 这一 Mathlib 中已有概念 (`RingTheory.Sequence.Isregular`)，它也使用了错误接口 (`IsRegularSequence`)。另一方面，Goedel 完全没有理解“regular sequence”和“system of parameters”的概念，生成了完全错误的陈述。

相比之下，Aria 的 Graph-of-Thought (GoT) 架构帮助它正确识别关键概念。它成功在 Mathlib 中搜索并找到 `RingTheory.Sequence.IsRegular`。通过 GoT 的分解能力，Aria 成功形式化了 system of parameters 的定义，并最终成功形式化整个猜想。

B ARIASCORER 的讨论与案例研究

通过把评估 grounding 到检索得到的 Lean 术语信息上，我们的检查器能够更准确地捕捉形式化陈述的精确语义，提升匹配和评估质量。下面展示若干例子，说明三项关键创新如何提升 AriaScorer 的性能。

B.1 隐含语义包含

非形式化陈述

$\mathbb{C}[x, y, z]/(x^2 + y^3 + z^7)$ 是 UFD。

条件与结论

条件:

1. 令 R 为环 $R = \mathbb{C}[x, y, z]/(x^2 + y^3 + z^7)$ 。

结论:

- R 是唯一分解整环 (UFD)。这意味着:
 1. R 是整环 (带有 $1 \neq 0$ 且没有零因子的交换环)。
 2. 对每个非零非单位元 $a \in R$ ，存在不可约元 $p_1, p_2, \dots, p_k \in R$ ，使得 $a = p_1 p_2 \cdots p_k$ 。
 3. 该分解唯一: 若 $a = q_1 q_2 \cdots q_m$ 是另一个不可约元分解，则 $k = m$ ，且存在 $\{1, 2, \dots, k\}$ 的置换 σ 和单位元 $u_1, u_2, \dots, u_k \in R$ ，使得对所有 i ， $p_i = u_i q_{\sigma(i)}$ 。

ARIA 翻译的形式化陈述

```

import Mathlib

noncomputable def kleinianSingularityIdeal : Ideal (MvPolynomial (Fin 3) ℂ) :=
  Ideal.span {
    (MvPolynomial.X 0 : MvPolynomial (Fin 3) ℂ)^2 +
    (MvPolynomial.X 1)^3 +

```

```

    (MvPolynomial.X 2)^7
  }

abbrev CxyzModKleinian := MvPolynomial (Fin 3) ℂ / kleinianSingularityIdeal

local instance : IsDomain CxyzModKleinian :=
  (Ideal.Quotient.isDomain_iff_prime kleinianSingularityIdeal).mpr sorry

theorem isUFD_of_C_xyz_mod_x2_y3_z7 :
  UniqueFactorizationMonoid CxyzModKleinian := by
  sorry

```

检索到的 LEAN 术语

```

{'name': ['MvPolynomial'], 'kind': 'definition', 'value': 'Type u_1 → (R : Type
u_2) → [inst : CommSemiring R] → Type (max u_1 u_2)', 'informal_name':
'Multivariate polynomials over  $R$  with variables in  $\sigma$ ',
'informal_description': 'The type of multivariate polynomials over a
commutative semiring  $R$  with variables indexed by a type  $\sigma$ . Each polynomial
is a finite formal linear combination of monomials, where a monomial is a
product of variables  $X_i$  for  $i \in \sigma$  raised to non-negative integer powers, with
coefficients in  $R$ .'}
{'name': ['Ideal', 'span'], 'kind': 'definition', 'value': '{ $\alpha$  : Type u} → [inst
: Semiring  $\alpha$ ] → Set  $\alpha$  → Ideal  $\alpha$ ', 'informal_name': 'Ideal generated by a
subset', 'informal_description': 'Given a subset  $s$  of a semiring  $\alpha$ , the ideal
span( $s$ ) is the smallest ideal of  $\alpha$  containing  $s$ , defined as the intersection
of all ideals containing  $s$ .'}

```

ARIASCRORER 的匹配与评估

LLM 完整输出:

逐项比较数学条件和结论与 Lean 4 形式化陈述:

1. 令 R 为环 $R = \mathbb{C}[x, y, z]/(x^2 + y^3 + z^7)$:
 - 数学: 研究对象是一个商环。基环是三变量多项式环 $\mathbb{C}[x, y, z]$, 系数在复数域中; 理想是由多项式 $x^2 + y^3 + z^7$ 生成的主理想。
 - Lean: `abbrev CxyzModKleinian := MvPolynomial (Fin 3) ℂ / kleinianSingularityIdeal`。这把该环定义为商环。基环 `MvPolynomial (Fin 3) ℂ` 表示以 $\mathbb{C}$ 为系数、变量由 `Fin 3` 索引的多变量多项式环, 分别对应 x, y, z 。理想 `kleinianSingularityIdeal` 定义为由集合中唯一多项式生成的 `Ideal.span`, 该多项式正对应 $x^2 + y^3 + z^7$ 。
 - 匹配: Perfectly match。
2. R 是唯一分解整环 (UFD):
 - 数学: 环 R 是整环, 并且每个非零非单位元都能唯一分解为不可约元乘积 (至多差一个顺序和单位因子)。
 - Lean: 结论分两部分形式化。首先, `local instance : IsDomain CxyzModKleinian` 断言该环是整环。其次, 主定理 `theorem isUFD_of_C_xyz_mod_x2_y3_z7 : UniqueFactorizationMonoid CxyzModKleinian` 断言唯一分解性质。在 Mathlib 中, 一个环是 UFD 当且仅当它同时满足 `IsDomain` 和 `UniqueFactorizationMonoid`。
 - 匹配: Perfectly match。

检查缺失条件/隐含条件:

- 数学陈述“ R 是 UFD”隐含 R 必须是整环。Lean 代码用 `local instance : IsDomain CxyzModKleinian` 显式给出这一点。这不是缺失条件, 而是对数学定义隐含部分的正确形式化。
- 匹配: Perfectly match。

分析

在该案例中，非形式化陈述断言商环 $R = \mathbb{C}[x, y, z]/(x^2 + y^3 + z^7)$ 是唯一分解整环 (UFD)，而 UFD 的定义要求 R 是整环。借助术语级检索，AriaScorer 能正确访问相关 Lean 术语定义，并跟踪新写入的局部实例。这使它能够识别 UFD 概念在形式代码中通过 `IsDomain` 和 `UniqueFactorizationMonoid` 的组合实现，尽管二者在代码中分开出现。

如果没有这种术语级语义 grounding，检查器可能会把新定义的局部实例误判为向结论添加了非平凡条件，而不是认识到它是 `UniqueFactorizationMonoid` 定义中隐含的前提。相反，AriaScorer 揭示了这一内部依赖并正确匹配该形式化。这一案例展示了语义 grounding 的力量：它使检查器超越表面形式，理解形式术语的结构意义及其相互关系，而纯文本比较无法做到这一点。

B.2 定义差异检测

非形式化陈述

设 $A, B \in \mathbb{Q}^\times$ 为有理数。考虑四元数环 $D_{A,B,\mathbb{R}} = \{a + bi + cj + dk \mid a, b, c, d \in \mathbb{R}\}$ ，其中乘法满足关系： $i^2 = A$ ， $j^2 = B$ ，且 $ij = -ji = k$ 。证明 $D_{A,B,\mathbb{R}}$ 作为 $\mathbb{R}$ -代数，要么同构于 $\mathbb{H}$ (Hamilton 四元数)，要么同构于 $\text{Mat}_{2 \times 2}(\mathbb{R})$ 。

条件与结论

条件：

1. $A, B \in \mathbb{Q}^\times$ ，即 A 和 B 是非零有理数。
2. $D_{A,B,\mathbb{R}}$ 是定义在集合 $\{a + bi + cj + dk \mid a, b, c, d \in \mathbb{R}\}$ 上的 $\mathbb{R}$ -代数。
3. $D_{A,B,\mathbb{R}}$ 中的乘法由下列关系定义： $i^2 = A$ ， $j^2 = B$ ， $ij = -ji = k$ 。

结论：

- $D_{A,B,\mathbb{R}}$ 作为 $\mathbb{R}$ -代数，同构于 $\mathbb{H}$ (Hamilton 四元数) 或同构于 $\text{Mat}_{2 \times 2}(\mathbb{R})$ (2×2 实矩阵代数)。可写作：

$$(D_{A,B,\mathbb{R}} \cong \mathbb{H}) \vee (D_{A,B,\mathbb{R}} \cong \text{Mat}_{2 \times 2}(\mathbb{R})),$$

其中 $\cong$ 表示 $\mathbb{R}$ -代数同构。

ARIA 翻译的形式化陈述

```
import Mathlib

theorem quaternion_algebra_over_reals_isomorphic_to_hamiltonians_or_matrices
  (A B : ℚ) (hA : A ≠ 0) (hB : B ≠ 0) :
  Nonempty (QuaternionAlgebra ℝ (A : ℝ) (B : ℝ) 1 ≅_a[ℝ] (QuaternionAlgebra ℝ
    (-1) (-1) 1)) ∨
  Nonempty (QuaternionAlgebra ℝ (A : ℝ) (B : ℝ) 1 ≅_a[ℝ] Matrix (Fin 2) (Fin
    2) ℝ) := by
  sorry
```

检索到的 LEAN 术语

```
{'name': ['Matrix'], 'kind': 'definition', 'value': "Type u → Type u' → Type v
  → Type (max u u' v)", 'informal_name': 'Matrix', 'informal_description':
  'The type of matrices with rows indexed by a type m, columns indexed by a
  type n, and entries in a type α. A matrix M can be represented as a
  function M : m → n → α, where M(i, j) gives the entry in row i and column j.'}
{'name': ['QuaternionAlgebra'], 'kind': 'structure', 'value': '(R : Type u_1) →
  R → R → R → Type u_1', 'informal_name': 'Quaternion Algebra',
  'informal_description': 'The structure representing a quaternion algebra over
  a commutative ring R with fixed coefficients a, b, c, where the elements are of
```

the form $x + yi + zj + wk$ with $x, y, z, w \in R$ and the multiplication rules are determined by $i^2 = a + bi$ and $j^2 = c$. The structure has four components: the real part ``re`` and three imaginary parts ``imI``, ``imJ``, ``imK`.``

ARIA 语义检查器的匹配与评估

逐项比较数学条件和结论与 Lean 4 形式化陈述:

1. $A, B \in \mathbb{Q}^\times$ (即 A 和 B 为非零有理数):

- 数学: $A, B \in \mathbb{Q}^\times$ 。
- Lean: $(A : \mathbb{Q}) (hA : A \neq 0) (hB : B \neq 0)$ 。
- 匹配: Perfectly match。

2. $D_{A,B,\mathbb{R}}$ 是定义在 $\{a + bi + cj + dk \mid a, b, c, d \in \mathbb{R}\}$ 上的 $\mathbb{R}$ -代数:

- 数学: 该代数的元素形式为 $a + bi + cj + dk$, 系数为实数。
- Lean: $\text{QuaternionAlgebra } \mathbb{R} (A : \mathbb{R}) (B : \mathbb{R}) 1$ 。
- 根据提供的 Mathlib 条目, QuaternionAlgebra 的元素形式为 $x + yi + zj + wk$, 其中 x, y, z, w 来自基环 R , 此处为 $\mathbb{R}$ 。因此元素形式匹配。
- 匹配: Perfectly match。

3. $D_{A,B,\mathbb{R}}$ 的乘法关系为 $i^2 = A$ 、 $j^2 = B$ 、 $ij = -ji = k$:

- 数学: 乘法规则为 $i^2 = A$ 和 $j^2 = B$ 。
- Lean: 乘法由 $\text{QuaternionAlgebra } \mathbb{R} (A : \mathbb{R}) (B : \mathbb{R}) 1$ 决定。
- 根据提供的 Mathlib 条目, $\text{QuaternionAlgebra } R a b c$ 具有规则 $i^2 = a + bi$ 和 $j^2 = c$ 。在 Lean 陈述中, R 为 $\mathbb{R}$, a 是 $(A : \mathbb{R})$, b 是 $(B : \mathbb{R})$, c 是 1。因此规则为 $i^2 = A + Bi$ 且 $j^2 = 1$, 与数学规则不匹配。
- 匹配: Major inconsistency。

4. $D_{A,B,\mathbb{R}}$ 作为 $\mathbb{R}$ -代数同构于 $\mathbb{H}$ 或 $\text{Mat}_{2 \times 2}(\mathbb{R})$:

- 数学: $(D_{A,B,\mathbb{R}} \cong \mathbb{H}) \vee (D_{A,B,\mathbb{R}} \cong \text{Mat}_{2 \times 2}(\mathbb{R}))$ 。
- Lean: $\text{Nonempty } (\text{QuaternionAlgebra } \mathbb{R} (A : \mathbb{R}) (B : \mathbb{R}) 1 \simeq_a [\mathbb{R}] (\text{QuaternionAlgebra } \mathbb{R} (-1) (-1) 1)) \vee \text{Nonempty } (\text{QuaternionAlgebra } \mathbb{R} (A : \mathbb{R}) (B : \mathbb{R}) 1 \simeq_a [\mathbb{R}] \text{Matrix } (\text{Fin } 2) (\text{Fin } 2) \mathbb{R})$ 。
- Lean 结论讨论的是 $\text{QuaternionAlgebra } \mathbb{R} (A : \mathbb{R}) (B : \mathbb{R}) 1$, 而如第 3 点所示, 这不是数学问题中的 $D_{A,B,\mathbb{R}}$ 。此外, Hamilton 四元数 $\mathbb{H}$ 满足 $i^2 = -1, j^2 = -1$, Lean 中写成 $\text{QuaternionAlgebra } \mathbb{R} (-1) (-1) 1$ 。但根据给定文档, 这对应规则 $i^2 = -1 - i$ 且 $j^2 = 1$, 也不是 $\mathbb{H}$ 。虽然 $\text{Matrix } (\text{Fin } 2) (\text{Fin } 2) \mathbb{R}$ 对 2×2 实矩阵的表示是正确的, 但整体陈述讨论的代数结构不同于数学问题。
- 匹配: Major inconsistency。

检查缺失条件/隐含条件:

- 没有缺失条件或隐含条件。
- 匹配: Perfectly match。

分析

在该案例中, 非形式化陈述给出 $D_{A,B,\mathbb{R}}$ 的乘法规则: $i^2 = A$ 、 $j^2 = B$ 、 $ij = -ji = k$ 。然而, 对应 Lean 术语的实际规则完全不同: $i^2 = A + Bi$, $j^2 = 1$ 。这些差异无法通过表面文本比较察觉, 因为非形式化和形式化版本都使用 A 和 B , 并具有相似的代数结构。通过术语级 grounding, AriaChecker 得到 QuaternionAlgebra 的显式定义, 从而使语义错误对 AriaScorer 变得明显。LeanScorer 会被表面文本相似性误导并错判该案例, 而 AriaScorer 正确标记为“Major Inconsistency”。这说明定义差异检测能帮助 AriaScorer 区分语义错误, 确保形式化陈述符合数学意图。

B.3 通过 GROUNDING 抑制幻觉

非形式化陈述

令 $\alpha = \sqrt{(2 + \sqrt{2})(3 + \sqrt{3})}$, 并考虑扩张 $E = \mathbb{Q}(\alpha)$ 。证明 $\text{Gal}(E/\mathbb{Q}) \cong Q_8$, 其中 Q_8 为 8 阶四元数群。

条件与结论

条件:

1. $\alpha = \sqrt{(2 + \sqrt{2})(3 + \sqrt{3})}$ 。
2. $E = \mathbb{Q}(\alpha)$, 即包含 $\mathbb{Q}$ 和 α 的最小域。

结论:

- 扩张 $E/\mathbb{Q}$ 的 Galois 群同构于 8 阶四元数群 Q_8 :

$$\text{Gal}(E/\mathbb{Q}) \cong Q_8.$$

ARIA 翻译的形式化陈述

```
import Mathlib

open Polynomial

/--
Let  $\alpha = \sqrt{(2 + \sqrt{2})(3 + \sqrt{3})}$ . The field  $E = \mathbb{Q}(\alpha)$  is a Galois
extension of  $\mathbb{Q}$ , and its Galois group is isomorphic to the quaternion group  $Q_8$ .
... Nonempty (P.Gal  $\cong$  QuaternionGroup 1) :=
sorry
```

检索到的 LEAN 术语

```
{'name': ['IntermediateField', 'adjoin'], 'kind': 'definition', 'value': '(F :
  Type u_1) → \n [inst : Field F] → {E : Type u_2} → [inst_1 : Field E] →
  [inst_2 : Algebra F E] → Set E → IntermediateField F E', 'informal_name':
  'Field adjunction of a set S to F', 'informal_description': 'Given a field
  extension E of F and a subset S ⊆ E, the structure
  `IntermediateField.adjoin F S` represents the intermediate field obtained by
  adjoining the elements hedral group; when n = 1, it is isomorphic to a cyclic
  group of order 4.'}
{'name': ['Polynomial'], 'kind': 'structure', 'value': '(R : Type u_1) → [inst :
  Semiring R] → Type u_1', 'informal_name': 'Univariate polynomials over a
  semiring', 'informal_description': 'The structure `Polynomial R` represents
  the type of univariate polynomials over a semiring `R`, denoted as `R[X]`.
  Polynomials are constructed using the variable `X` and constant embedding `C`
  from `R`, forming a semiring structure.'
```

ARIA 语义检查器的匹配与评估

逐项比较数学条件和结论与 Lean 4 形式化陈述:

1. 条件: $\alpha = \sqrt{(2 + \sqrt{2})(3 + \sqrt{3})}$:
 - 数学: α 由给定公式构造性定义, 指定了一个特定 (正实) 数。
 - Lean: let P : $\mathbb{Q}[X]$:= $X^8 - 24 * X^6 + 144 * X^4 - 288 * X^2 + 144$ 以及 $\exists \alpha \in \text{rootSet } P \text{ P.SplittingField}$, ...。这没有使用 sqrt 公式定义 α , 而是断言存在一个特定多项式 P 的根 α 。虽然 P 是数

学中 α 的最小多项式，但形式化陈述谈的是 P 的任意根，而不是题目中的具体数。

- 匹配：Minor Inconsistency。

2. 条件： $E = \mathbb{Q}(\alpha)$ ：

- 数学： E 定义为域扩张 $\mathbb{Q}(\alpha)$ 。题目随后要求计算该扩张的 Galois 群，这隐含或要求 $E/\mathbb{Q}$ 为 Galois 扩张。
- Lean： `IntermediateField.adjoin  $\mathbb{Q} \{ \alpha \} = \top$` 。术语 `IntermediateField.adjoin  $\mathbb{Q} \{ \alpha \}$` 对应 $\mathbb{Q}(\alpha)$ 。 $\top$ 指环境域，即 `P.SplittingField`。因此，该陈述断言 $\mathbb{Q}(\alpha)$ 是 P 的分裂域。这把数学中隐含的 $E/\mathbb{Q}$ 为 Galois 的假设转化为定理中的显式强条件。
- 匹配：Minor Inconsistency。

3. 结论： $\text{Gal}(E/\mathbb{Q}) \cong Q_8$ ：

- 数学：扩张 $E/\mathbb{Q}$ 的 Galois 群同构于 8 阶四元数群 Q_8 。
- Lean： `Nonempty (P.Gal  $\simeq$  * QuaternionGroup 1)`。`P.Gal` 对应 $\text{Gal}(E/\mathbb{Q})$ 。`Nonempty (G  $\simeq$  * H)` 表示 G 与 H 同构。然而，`QuaternionGroup 1` 是参数 $n = 1$ 的广义四元数群，它同构于 4 阶循环群 (`C_4`)。8 阶四元数群 Q_8 对应 `QuaternionGroup 2`。
- 匹配：Major Inconsistency。

检查缺失条件/隐含条件：

- Lean 陈述引入了多项式 P 的显式定义，这在原始数学问题中没有出现，但它是证明所需的中间结果。这是附加条件。没有缺失条件。
- 匹配：Minor Inconsistency。

去掉术语级 GROUNDING 时的误判

逐项比较数学条件和结论与 Lean 4 形式化陈述：

1. α 的定义：

- 数学： $\alpha = \sqrt{(2 + \sqrt{2})(3 + \sqrt{3})}$ ，定义了一个具体正实数。
- Lean：用多项式 P 及 $\exists \alpha \in \text{rootSet } P$ `P.SplittingField`, ... 隐式把 α 定义为该多项式的任意根。这是从具体定义转向抽象代数定义。
- 匹配：Minor Inconsistency。

2. 域 E 的定义：

- 数学： $E = \mathbb{Q}(\alpha)$ ，即向 $\mathbb{Q}$ 添加具体数 α 得到的域。证明该域是 $\mathbb{Q}$ 上的 Galois 扩张是证明的一部分。
- Lean：环境域是多项式 P 的分裂域。条件 `IntermediateField.adjoin  $\mathbb{Q} \{ \alpha \} = \top$` 表明该分裂域由单个根 α 生成。这重构了域的定义：问题被放在 P 的分裂域中，并且要证明它可以由单个根生成。
- 匹配：Minor Inconsistency。

3. 被比较的群：

- 数学： $\text{Gal}(E/\mathbb{Q})$ 和 Q_8 。
- Lean： `P.Gal` 和 `QuaternionGroup 1`。`P.Gal` 是 `P.SplittingField` 在 ‘ $\mathbb{Q}$ ’ 上的 Galois 群，对应 $\text{Gal}(E/\mathbb{Q})$ 。`docstring` 和上下文暗示 `QuaternionGroup 1` 是 Q_8 的名称。
- 匹配：Perfectly match。

4. 结论：

- 数学： $\text{Gal}(E/\mathbb{Q}) \cong Q_8$ 。
- Lean： `Nonempty (P.Gal  $\simeq$  * QuaternionGroup 1)`，表示 `P.Gal` 与 `QuaternionGroup 1` 之间存在群同构。`Nonempty wrapper` 是 Lean 中表达命题存在性的标准方式。

- 匹配：Perfectly match。

检查缺失条件/隐含条件：

- 数学问题隐含需要找到 α 的最小多项式，并证明扩张 $\mathbb{Q}(\alpha)/\mathbb{Q}$ 是 Galois (即该多项式的分裂域)。Lean 陈述通过一开始提供多项式 p ，并加入 `IntermediateField.adjoin  $\mathbb{Q} \{ \alpha \} = \top$` 条件，把这些方面显式化。形式化陈述更显式，这是形式化的特征，不是条件缺失。
- 匹配：Perfectly match。

分析

在该案例中，Lean 陈述使用关键术语 `QuaternionGroup 1` 来声称同构 `Nonempty (P.Gal  $\simeq^* \text{QuaternionGroup } 1$ )`。人们很容易假设 `QuaternionGroup 1` 指 8 阶四元数群 Q_8 。实际上，`QuaternionGroup 1` 同构于循环群 C_4 ，而 Q_8 的实际表示是 `QuaternionGroup 2`。当 LLM 只基于表面文本进行检查时，这种细微但重要的区别会被忽略。在没有 Lean 术语信息的设置下，检查器被 LLM 的幻觉误导，给出错误判断。相比之下，AriaScorer 将检查流水线 grounding 到具体语义。借助 `QuaternionGroup n` 的定义，AriaScorer 正确地把该陈述标为“Major Inconsistency”，与人工标注一致。这展示了 hallucination suppression 如何把 LLM 推理约束在 Lean 术语内部，从而保证语义正确性检查的精确性。

C 消融研究

为量化 Aria 智能体核心组件的个体贡献，我们进行了一系列综合消融研究。我们系统地禁用 Reflection、Graph-of-Thought (GoT) 和 Retrieval-Augmented Generation (RAG) 模块，以衡量它们对性能的影响。所有实验都在挑战性基准 FATE-X 和 homological conjectures 上进行，结果见表 3 和表 4。

表 3: Conjectures 数据集上的消融研究结果。移除 Aria 的关键组件会显著降低性能，凸显各组件的个体贡献。所有数值均为成功率 (%)。

配置	Final acc.
Aria (完整系统)	42.9
Aria 消融：	
without Reflection	0
without GoT	7.1
without RAG	0
Baseline (Gemini)	0

表 4: FATE-X 基准上的消融研究结果。所有数值均为成功率 (%)。

配置	Compiler	Final acc.
Aria (完整系统)	69.0	44.0
Aria 消融：		
without Reflection	19.0	14.0
without GoT	69.0	38.0
without RAG	61.0	43.0
Baseline (Gemini)	27.0	21.0

C.1 消融 REFLECTION 机制

该研究旨在量化智能体核心迭代自纠错机制的贡献。在完整 Aria 智能体中，每个生成步骤 (无论是前置定义还是最终定理) 都嵌入一个 refinement loop，允许 16 次反思尝试。在该循环内，智能体生成候选形式定义或陈述，接收编译器反馈，并利用该反馈指导下一次生成尝试。

在消融中，我们完全禁用 refinement loop，将智能体限制为每个阶段只有一次生成尝试。

如表 4 所示，消融 reflection 模块会使 FATE-X 上最终准确率从 44% 降至 14%，编译成功率从 69% 降至 19%，甚至低于基线。在 Conjectures 数据集上也观察到类似的显著下降，成功率从 42.9% 降至 0%。结果表明，单次生成通常不足以同时捕捉复杂数学陈述的语义细微性和句法严格性。因此，我们认为 Reflection 模块是智能体架构中的关键部分。

C.2 消融 GoT 规划器

该消融实验设置如下：首先从原始非形式化陈述中抽取一个扁平的概念关键词列表。随后，对列表中的每个概念，使用 LeanSearch 在 Mathlib 中检索。与完整系统不同，该过程无论搜索结果如何，都不再进行递归分解。智能体随后只使用这次搜索的结果直接合成最终形式化陈述。

如表 3 所示，在 Conjectures 数据集上，完整 Aria 系统成功形式化了 14 个猜想中的 6 个，而没有 GoT 的版本只成功 1 个。

表 4 表明，在挑战性 FATE-X 基准上，没有 GoT 的 Aria 仍保持 69.0% 的编译成功率，但最终准确率从 44.0% 降至 38.0%。为进一步研究 GoT 对翻译质量的影响，我们在 FATE-H 基准上进行了额外测试。

表 5: 完整 Aria 智能体与 GoT 消融版本在 FATE-H 基准上的性能比较。所有数值均为成功率 (%)。

配置	Compiler	Final acc.
Aria (完整系统)	89.0	71.0
Aria (without GoT)	95.0	54.0

表 5 的结果揭示了一个在较简单 FATE-H 数据集上的反直觉现象：消融智能体取得更高编译成功率 (95%)。然而，尽管它产出更多可编译结果，最终准确率 (56%) 仍低于完整系统 (71%)。

这些实验揭示了 GoT 模块的关键作用。在多数概念已存在于 Mathlib 或依赖简单的较简单问题上，GoT 模块的贡献并不显著。尽管如此，它仍保证这些问题的形式化更稳定。然而，在真实世界的猜想级数据集（如 conjecture）上，它的重要性被放大。这是因为这些问题要求智能体形式化 Mathlib 中不存在的新颖高层概念。没有 GoT 时，智能体试图单体生成解决方案，会出现两种不同失败模式：第一是合成失败，即智能体无法正确生成复杂定义；第二是接口幻觉。

因此，GoT 是猜想自动形式化所需的创造性数学构造中不可或缺的推理引擎。此外，它支持模块化形式化风格，使自动形式化器能够自主书写定义。

C.3 消融 RAG 模块

为衡量 Retrieval-Augmented Generation (RAG) 的价值，我们设计该研究，以比较基于实时工具的检索与依赖大语言模型 (LLM) 预训练静态知识的差异。

在完整系统中，智能体的 grounding 过程通过 LeanSearch 执行。LLM 的任务被限制为在这一经过验证的选项集合上推理。对于消融版本，我们完全禁用检索工具。相反，智能体直接询问 LLM，让其根据自身知识回忆某个概念的正确形式名称。

表 3 和表 4 中的消融研究揭示了 RAG 模块的关键作用，尤其是在问题复杂度增加时。虽然在 FATE-X 上，消融版本只导致最终准确率中等幅度下降（从 69% 到 61%），但在更具挑战性的 Conjectures 数据集上，其影响是绝对的，成功率从 42.9% 崩塌到 0%。

这种差异揭示了智能体能力的一项关键洞见。对于 FATE-X 中等复杂任务，智能体可以通过强大的自反思机制部分弥补缺失检索的问题。通过解释编译器关于“unknown identifiers”的精确反馈，智能体可能迭代地重新发现正确的 Mathlib 定义。然而，这种试错恢复过程不足以应对复杂猜想。0% 准确率说明，没有 RAG 提供的上下文 grounding，LLM 对 Mathlib 的不准确内部知识会导致它幻觉不存在的定义，并自信地判断它们已被 grounding。这一基础性错误会阻止可编译代码生成，表明 RAG 模块对于挑战性数学推理任务的成功至关重要。

D 关于使用大语言模型（LLM）的声明

根据政策，我们披露大语言模型（LLM）在本文稿准备过程中发挥了重要作用。作者对所有内容承担全部责任，包括这些模型生成的任何文本，并已仔细审阅和编辑所有输出，以保证准确性、原创性和科学完整性。

我们使用 Google Gemini-2.5-Pro 作为语言编辑工具。其作用严格限于提高清晰度、纠正语法错误和改写句子。